- **Thread vs Process**
- **System Call**

Dr. Manmath N. Sahoo
Dept. of CSE, NIT Rourkela

Threads Vs Processes

```
1.  for(i=0;i<100000;i++)
2.  {
3.          receive data from i/p device
            //RcvData()
4.          send data on communication link
            //SendonLink()
5.  }
```

- If communication link is not free then the process will halt in line#4. Even though i/p device is free, it can be used.
- In line#3, if user has not specified any input then communication link may be underutilized.
  - If there is no other process available, then CPU can't make a context switch even (at line#3)

Threads Vs Processes

Solution

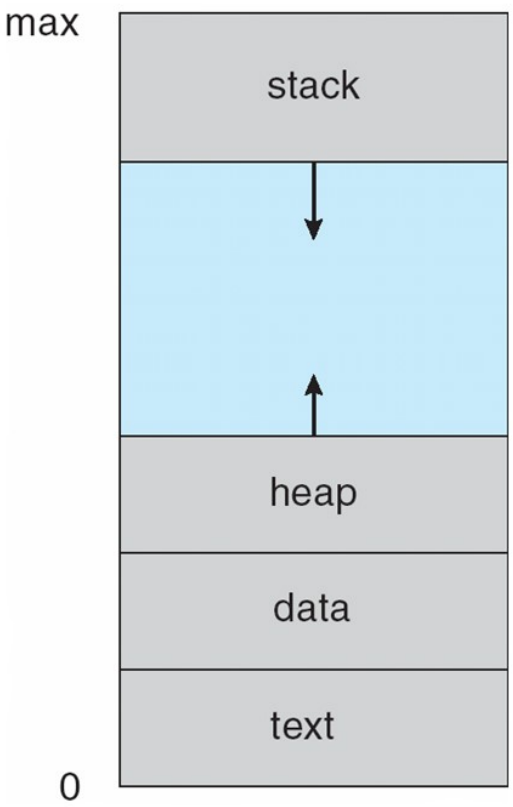
- Consider RcvData() and SendonLink() to be two different entities
- RcvData(), on data availability, will put data in a queue
- SendonLink(), on link availability, will send data from the queue
- Whoever is free, can execute on CPU

RcvData() and SendonLink() entities can be implemented as processes or Threads

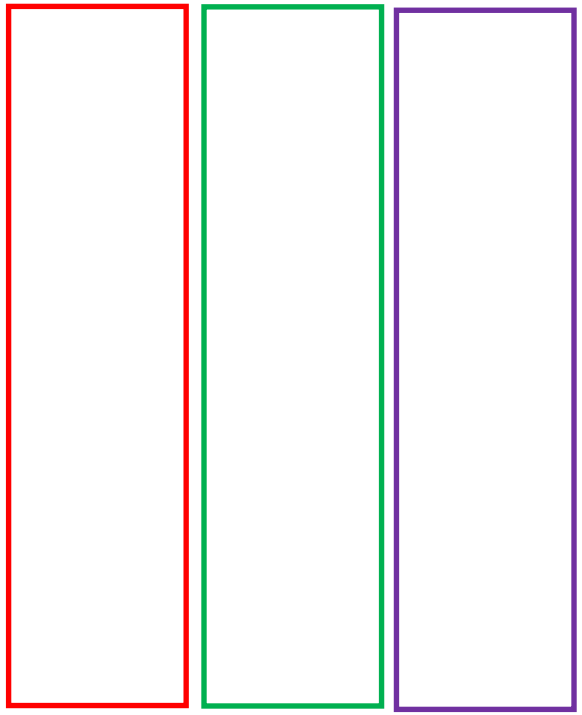
Threads Vs Processes

- Threads are the unit of execution in a process.
- A thread shares address space with it's parent.
- Per Thread items
  - Thread ID – Unique identifier
  - Program Counter – which instruction to execute next
  - Registers – for computation
  - Stack – contains the execution history
  - State – thread state

address space of a process



Single and Multithreaded Processes



Threads Vs Processes

- Threads share the address space of the process that created it; processes have their own address space.
- Threads have direct access to the data segment of its process; processes have their own copy of the data segment of the parent process.
- Threads can directly communicate with other threads of its parent; processes must use interprocess communication to communicate with sibling processes.
- Threads have almost no context switch overhead; processes have considerable overhead.
- New threads are easily created; new processes require duplication of the parent process.

It is a light weight process and faster

Thread Benefits

- **Responsiveness –** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing
- **Economy –** cheaper than process creation, thread switching lower overhead than context switching
- **Scalability –** threads can take advantage of multiprocessor architectures

Thread: Programmer's View

```
void fn1(int arg0, int arg1, …) {…}
void fn2(int arg0, int arg1, …) {…}
main() {
      …
      tid1 = CreateThread(fn1, arg0, arg1, …);
      Print("Main thread running");
      tid2 = CreateThread(fn1, arg0, arg1, …);
      …
}
```

- At the point CreateThread is called, execution continues in parent thread in main function, and execution starts at fn1 in the child thread, both in parallel

How Thread Can Help? – Example

- Consider the following code fragment

```
for(k = 0; k < n; k++)
     a[k] = b[k] * c[k] + d[k] * e[k];
```

- Rewrite this code fragment as:

```
CreateThread(fn, 0, n/2);
CreateThread(fn, n/2, n);
fn(l, m) {
     for(k = l; k < m; k++)
          a[k] = b[k] * c[k] + d[k] * e[k];
}
```

- What did we gain?

System Call

- A request by an active process/thread to the Kernel for a service
- Defines the interface between the user and the OS
- The process switches to Kernel mode from user mode, during a system call
- Preemptive vs. non-preemptive kernel
  - NonPreemptive: Linux 2.4 Kernel
  - Preemptive: Linux 2.6 Kernel

read system call – `read(fd,&buffer,nbytes)`